

RETAIL DEMAND FORECASTING

STUDENT 3 — CONTRIBUTION DOCUMENTATION

Spark / PySpark Processing, Analytics & Demand Forecasting

Item	Details
Project	Retail Demand Forecasting
Student Role	Student 3 — Spark / PySpark
Student name	Akhila Vaidya
Input from Student 2	Structured data layer: Hive external table retail_analytics.retail_demand over HDFS /data/retail/raw
Environment	Google Colab (PySpark in local mode); files stored under /content/retail
Spark version	4.0.4 (printed by the notebook)
Records / Columns	76,000 / 16
Target variable	Demand

This document covers Student 3's individual contribution: reading the retail dataset with Spark, checking data quality, engineering features, running distributed analytics, building a demand-forecasting model with Spark MLlib, and saving the results for Student 4 and Student 5.

How to read this document. The results in this document come from the notebook run in Google Colab. Yellow items in square brackets are still to be added: screenshots, and your own issues and fixes.

1. Student 3 Contribution — Overview

Student 1 stored the raw CSV in HDFS and Student 2 exposed it through Hive for SQL analytics. Student 3 adds the distributed processing and analytics layer: Spark reads the same dataset, cleans and enriches it, answers further business questions, and trains a model that forecasts demand.

Stage	Student 1	Student 2	Student 3
Main role	Data ingestion + HDFS	Hive + data modeling	Spark / PySpark + forecasting
Input	Local CSV	Raw CSV in HDFS	Same CSV (Hive schema as reference)
Technology	Hadoop HDFS	Hive / Beeline	Spark, PySpark, Spark SQL, MLlib
Main output	HDFS raw file	External Hive table + SQL results	Cleaned Parquet data, result tables, forecasting model

2. Objective and Analytics Questions

Objective: process the 76,000-record retail dataset with distributed Spark, quantify the factors associated with demand, and build a model that predicts demand for a store-product-day.

- Which stores perform best and where is demand being left unmet?
- How does demand change over time (quarterly trend)?
- How large is the promotion lift in each category?
- How much lower is demand during epidemic periods?

- How does the discount level relate to demand?
- Which products lead demand in each region?
- Can past demand and known business factors forecast future demand?

3. Input from Student 2 (Hand-off)

Hand-off item	Value
HDFS raw path (project)	/data/retail/raw/retail_demand.csv
Hive database / table	retail_analytics / retail_demand (external table)
Record count verified	76,000
Hive column types	date, promotion, epidemic stored as STRING; discount as DOUBLE
Student 3 handling	Spark applies an explicit schema: date to DATE, promotion / epidemic / discount to INT

Student 3 used the same project dataset and the same column names as Student 2. The CSV was uploaded to a Google Colab runtime, where Spark reads it from /content/retail_demand.csv. The link between the stages is the shared dataset and schema, not a network connection between separate systems.

4. Spark Environment

Component	Details
Platform	Google Colab (cloud notebook)
Spark	PySpark 4.0.4, installed with pip
Spark mode	Local mode: master = local[*] (all cores of the Colab machine)
Input partitions	2 (shown by df.rdd.getNumPartitions())
Storage	Colab file system under /content (stands in for HDFS)
Project environment	The project Docker stack (HDFS, Hive, Spark master and worker, Jupyter) runs Spark 3.1.1; the same code runs there by changing only the three path variables

HDFS was not used for this stage because the Hadoop Docker stack was not set up on the machine used by Student 3. Student 1 and Student 2 demonstrate HDFS and Hive; Student 3 demonstrates the Spark processing logic, which is identical in either environment.

5. Reading the Data with Spark

A SparkSession is created and the CSV is read with an explicit schema instead of inferSchema. This avoids an extra pass over the data and guarantees correct types. Column names are converted to snake_case to match Student 2 Hive table.

```
spark = SparkSession.builder.appName("RetailDemandForecasting_Student3").getOrCreate()
raw = (spark.read.option("header", True).schema(schema)
      .csv("/content/retail_demand.csv"))
for c in raw.columns:
    df = df.withColumnRenamed(c, c.strip().lower().replace(" ", "_"))
print(df.count())      # 76000
```

Result: Spark version 4.0.4, master local[*], 76,000 records, 16 columns, 2 partitions. Evidence: root

```

|-- date: string (nullable = true)
|-- store_id: string (nullable = true)
|-- product_id: string (nullable = true)
|-- category: string (nullable = true)
|-- region: string (nullable = true)
|-- inventory_level: integer (nullable = true)
|-- units_sold: integer (nullable = true)
|-- units_ordered: integer (nullable = true)
|-- price: double (nullable = true)
|-- discount: integer (nullable = true)
|-- weather_condition: string (nullable = true)
|-- promotion: integer (nullable = true)
|-- competitor_pricing: double (nullable = true)
|-- seasonality: string (nullable = true)
|-- epidemic: integer (nullable = true)
|-- demand: integer (nullable = true)

```

Record count : 76000

Partitions : 2

date	store_id	product_id	category	region	inventory_level	units_sold	units_ordered	price	discount	weather_condition	promotion	competitor_pricing	seasonality	epidemic	demand
2022-01-01	S001	P0001	Electronics	North	195	102	252	72.72	5	Snowy					
0	85.73	Winter	0	115											
2022-01-01	S001	P0002	Clothing	North	117	117	249	80.16	15	Snowy					
1	92.02	Winter	0	229											
2022-01-01	S001	P0003	Clothing	North	247	114	612	62.94	10	Snowy					
1	60.08	Winter	0	157											
2022-01-01	S001	P0004	Electronics	North	139	45	102	87.63	10	Snowy					
0	85.19	Winter	0	52											
2022-01-01	S001	P0005	Groceries	North	152	65	271	54.41	0	Snowy					
0	51.63	Winter	0	59											

only showing top 5 rows

6. Data Quality Checks

Check	Result
Records / columns	76,000 / 16
Null values in any column	0
Fully duplicated rows	0
Duplicate (date, store_id, product_id) keys	0
Negative demand, units or non-positive price	0
promotion / epidemic values outside 0 and 1	0
Date range	2022-01-01 to 2024-01-30 (760 days)
Series structure	5 stores × 20 products = 100 daily series of 760 records each
Demand range / mean	4 to 430 / 104.32

Observations from profiling

- units_ordered is 0 on about 62% of records, so orders are placed only on some days.
- A product_id appears under more than one category (up to five), so product-level analysis in this project uses product_id on its own.
- Each store belongs to exactly one region; S001 and S005 are both in the North region.
- No cleaning was needed to remove records; the dataset passed all checks.

7. Feature Engineering

Feature	Definition	Purpose
year, quarter, month, day_of_week, week_of_year	Extracted from date	Time patterns and trend
demand_gap	demand – units_sold	Unmet demand (positive) or surplus sales
price_vs_competitor	price – competitor_pricing	Relative price position
effective_price	price × (1 – discount/100)	Price paid after discount
demand_lag_1, demand_lag_7	Demand 1 and 7 days earlier (Window lag)	Recent demand history
demand_roll7_avg, demand_roll28_avg	Average of previous 7 / 28 days	Short and long-term demand level

```
w = Window.partitionBy("store_id","product_id").orderBy("date")
df = (df.withColumn("demand_lag_1", F.lag("demand",1).over(w))
      .withColumn("demand_roll7_avg",
                  F.avg("demand").over(w.rowsBetween(-7, -1))))
```

The rolling windows end at the previous row (–1), so the current day's demand is never used to predict itself. The DataFrame is cached because every later analysis reuses it, and the cleaned layer is saved as Parquet in /data/retail/processed/retail_clean.

8. Distributed Analytics Results

These analyses complement Student 2's Hive queries (category, region with seasonality, weather with promotion) by answering different questions. The values below are descriptive patterns in the data, not proof of cause and effect.

A1. Store performance

Store	Region	Records	Total demand	Avg demand	Avg demand gap
S002	South	15,200	1,625,471	106.94	15.75
S003	East	15,200	1,618,324	106.47	15.97
S005	North	15,200	1,607,085	105.73	16.07
S001	North	15,200	1,547,573	101.81	15.09
S004	West	15,200	1,529,651	100.63	14.57

S002 (South) has the highest total demand and S004 (West) the lowest, but the spread is small: about 6% between the highest and lowest store.

A2. Quarterly demand trend

Year	Quarter	Total demand	Avg demand
2022	Q1	1,014,600	112.73
2022	Q2	889,913	97.79
2022	Q3	954,437	103.74
2022	Q4	1,003,702	109.10
2023	Q1	891,357	99.04
2023	Q2	907,494	99.72
2023	Q3	1,062,615	115.50
2023	Q4	909,827	98.89
2024	Q1	294,159	98.05

Average demand peaks in 2022 Q1 (112.73) and 2023 Q3 (115.50). 2024 Q1 contains only January, so its total is not comparable with full quarters.

A3. Promotion lift by category

Category	Avg demand (no promo)	Avg demand (promo)	Lift %
Electronics	88.58	115.65	30.6%
Furniture	66.86	87.28	30.5%
Toys	84.34	109.47	29.8%
Groceries	110.30	142.76	29.4%
Clothing	102.72	132.81	29.3%

Promotions are associated with roughly 29% to 31% higher demand in every category, so the effect is consistent across categories.

A4. Epidemic impact

Epidemic flag	Records	Avg demand
0 (no epidemic)	60,800	112.86
1 (epidemic)	15,200	70.16

Average demand is about 38% lower in epidemic records (70.16 vs 112.86), the largest single difference found in the dataset.

A5. Discount level vs demand

Discount %	Records	Avg demand
0%	17,126	94.75
5%	16,918	94.94
10%	23,298	102.77
15%	6,245	123.58
20%	6,191	124.07
25%	6,222	122.97

Demand is about 95 at 0% and 5% discount, 103 at 10%, and about 123 to 124 at 15% to 25%. The gain levels off after 15%, so deeper discounts add little extra demand in this data.

A6. Unmet demand by category

Category	Avg demand gap	% records where demand > units sold
Groceries	18.10	70.71%
Clothing	17.98	72.58%
Electronics	14.44	70.50%
Toys	14.16	70.55%
Furniture	9.21	67.11%

Groceries and Clothing have the largest average shortfall between demand and sales (about 18 units), which makes them the first candidates for review of stock availability.

A7. Top 3 products per region (window ranking)

Region	Rank	Product	Total demand
East	1	P0002	93,629
East	2	P0007	92,857
East	3	P0019	92,759
North	1	P0018	185,747
North	2	P0009	184,218
North	3	P0007	183,003
South	1	P0013	93,392
South	2	P0010	93,270
South	3	P0008	92,425
West	1	P0001	93,465

Region	Rank	Product	Total demand
West	2	P0002	93,222
West	3	P0004	93,043

North shows roughly double the product totals of other regions because two stores (S001 and S005) are in that region.

```
w_rank = Window.partitionBy("region").orderBy(F.desc("total_demand"))
top = (prod_region.withColumn("rank", F.row_number().over(w_rank))
      .filter("rank <= 3"))
```

9. Spark SQL

The cleaned DataFrame is registered as a temporary view so SQL can run on Spark. This shows how Student 2 Hive-style SQL translates to Spark.

```
df.createOrReplaceTempView("retail_clean")
spark.sql("""SELECT weather_condition, ROUND(AVG(demand),2) avg_demand,
          ROUND(AVG(demand_gap),2) avg_demand_gap
          FROM retail_clean GROUP BY weather_condition
          ORDER BY avg_demand DESC """).show()
```

Weather	Avg demand	Avg demand gap
Sunny	115.17	18.14
Cloudy	105.40	15.78
Rainy	95.11	13.13
Snowy	94.05	13.10

Sunny weather has the highest average demand and snowy weather the lowest, a difference of about 21 units.

10. Saving Results

Spark saved the cleaned data and every result table. In Google Colab they are stored under /content/retail. In the project Hadoop environment the same code writes to the HDFS paths in the second column when the three path variables in the configuration cell are changed.

Colab path	Project HDFS path	Content
/content/retail/processed/retail_clean	/data/retail/processed/retail_clean	Cleaned data with features (Parquet)
/content/retail/results/store_summary	/data/retail/results/store_summary	A1 store performance
/content/retail/results/quarter_trend	/data/retail/results/quarter_trend	A2 quarterly trend
/content/retail/results/promotion_lift	/data/retail/results/promotion_lift	A3 promotion lift
/content/retail/results/epidemic_impact	/data/retail/results/epidemic_impact	A4 epidemic impact
/content/retail/results/discount_effect	/data/retail/results/discount_effect	A5 discount effect
/content/retail/results/unmet_demand	/data/retail/results/unmet_demand	A6 unmet demand
/content/retail/results/top_products_per_region	/data/retail/results/top_products_per_region	A7 top products
/content/retail/results/model_metrics	/data/retail/results/model_metrics	Model comparison

Colab path	Project HDFS path	Content
/content/retail/results/rf_predictions	/data/retail/results/rf_predictions	Random Forest predictions
/content/retail/results/rf_feature_importance	/data/retail/results/rf_feature_importance	Top Random Forest features

```
!ls -R /content/retail/results
```

Important. Colab deletes /content when the session ends. Before closing, download the results and share them with Student 4 and Student 5:

```
!zip -r /content/retail_results.zip /content/retail
from google.colab import files
files.download("/content/retail_results.zip")
```

11. Demand Forecasting with Spark MLlib

Design decisions

- Target: demand for a store-product-day.
- Features: category, region, store, product, weather, seasonality, price, discount, promotion, competitor pricing, epidemic flag, calendar fields, and the lag and rolling demand features.
- Excluded: units_sold, units_ordered and inventory_level, because they are recorded on the same day as demand and would not be known when forecasting (target leakage).
- Split: time-based. Training uses 2022-01-01 to 2023-09-30 (63,100 rows) and testing uses 2023-10-01 to 2024-01-30 (12,200 rows), because shuffling would let the model see the future.
- Models: a naive baseline (yesterday demand), Linear Regression and Random Forest (50 trees, depth 8, seed 42).
- Pipeline: StringIndexer, OneHotEncoder, VectorAssembler, then the regressor.

Results on the test period

Model	RMSE	MAE	R ²
Baseline (lag-1)	51.465	39.671	-0.312
Linear Regression	32.790	25.374	0.467
Random Forest	31.931	24.715	0.495

Interpretation

- Random Forest is the best model. Its RMSE is about 38% lower than the baseline (51.47 to 31.93) and about 3% lower than Linear Regression.
- The baseline has a negative R², which means yesterday demand predicts worse than simply using the average. Daily demand is noisy, so a good model must use the wider context.
- An R² of 0.495 means the model explains about half of the variation in daily demand. The rest is day-to-day randomness that these features cannot capture, so the forecasts show the right level but not every spike.
- Both models beat the baseline by a large margin, so the engineered features add real value.

Most important features (Random Forest)

Feature	Importance
demand_roll7_avg	0.252
epidemic	0.141
promotion	0.134
demand_roll28_avg	0.085
category = Groceries	0.081
price	0.051
discount	0.045
category = Furniture	0.041

The 7-day average of past demand is the strongest signal, followed by the epidemic flag and promotion, which agrees with the analytics in Section 8. `demand_lag_1` has low importance (0.017) because the rolling averages carry the same information in a smoother form.

Be careful when explaining. The model shows association, not proof that promotions or epidemic periods cause changes in demand.

12. Why Spark? Spark vs Hive

Topic	Hive (Student 2)	Spark (Student 3)
Main use	SQL layer and tables over HDFS data	Transformations, window functions, machine learning
Interface	HiveQL via Beeline	Python DataFrame API, Spark SQL, MLlib
Processing	Queries compiled to jobs (MapReduce in this setup)	In-memory distributed processing with DAG optimization
Reuse of data	Each query reads the table again	<code>cache()</code> keeps a DataFrame in memory across analyses
Best for	Reporting and aggregation	Feature engineering and model training

Spark is used instead of only local Pandas because the same code runs unchanged on a cluster: data is split into partitions, transformations are lazy and optimized as a DAG, and actions such as `count()` or `write()` trigger distributed jobs. If the dataset grew from 76,000 to 100 million rows, the code would stay the same, with more workers, more partitions, Parquet storage and partitioning the output by date.

13. Complete Flow: Student 1, 2 and 3

```
Student 1: Local CSV -> HDFS /data/retail/raw/retail_demand.csv
                        |
Student 2:              +--> Hive external table (SQL analytics)
                        |
Student 3: same CSV ----->+--> Spark read (explicit schema), Google Colab
                               -> quality checks
                               -> feature engineering (window functions)
                               -> analytics A1-A7 + Spark SQL
                               -> MLlib forecasting model
                               -> /content/retail/processed and /results
```

14. Hand-off from Student 3

Hand-off item	Value
Cleaned data	/content/retail/processed/retail_clean (Parquet)
Result tables	/content/retail/results/* (CSV, one folder per table), shared as retail_results.zip
Key findings	Promotion lift about 30%; epidemic periods about 38% lower; discount gains level off after 15%; Random Forest R^2 0.495
Code	student3_retail_spark_analysis_COLAB.ipynb
Useful for next stage	Summary tables suit lookup-style storage and presentation charts

15. Formal Contribution Statement

You can use this paragraph in the final project report:

Student 3 was responsible for the Spark / PySpark processing and analytics layer of the Retail Demand Forecasting project. Working in a Google Colab PySpark environment (local mode), Student 3 read the 76,000-record, 16-column retail_demand dataset using an explicit schema and verified data quality. Time features, unmet-demand measures, price features and lag and rolling demand features were engineered with Spark window functions, and the cleaned dataset was saved as Parquet. Seven distributed analyses covering stores, quarterly trend, promotion lift, epidemic impact, discount effect, unmet demand and top products per region were produced and saved, together with a Spark SQL query. A Spark MLlib forecasting pipeline was trained on a time-based split and compared with a naive baseline; the Random Forest performed best (RMSE 31.93, R^2 0.495). The outputs were handed to Student 4 and Student 5 for the advanced-technology and integration stages.

16. Issues Encountered and Resolutions

Issue	Resolution
The Hadoop / Docker stack was not available on the machine used for Student 3	Ran the same PySpark code in Google Colab in local mode; documented the equivalent HDFS paths
Colab has no HDFS and its files are deleted when the session ends	Saved outputs under /content/retail and downloaded them as a zip for the team

Edit this table so it lists only the problems you actually faced.

17. Evidence Produced by Student 3

- Notebook running successfully in Google Colab (all cells executed).
- Spark version and master output (4.0.4, local[*]).
- Schema and 76,000-row count.
- Data quality check output.
- Feature engineering sample rows.
- Analytics tables A1 to A7 and the Spark SQL result.
- File listing of /content/retail/processed and /content/retail/results.

- Model metrics and feature importance.

18. Viva / Presentation Explanation

What was your role?

I was responsible for Spark / PySpark processing. I read the retail data from HDFS, checked quality, created features, ran distributed analytics, built a forecasting model and saved the outputs to HDFS.

Why Spark when Hive already exists?

Hive gives a SQL layer. Spark adds in-memory distributed processing, window functions, caching and MLlib, which I needed for feature engineering and forecasting.

What happens when your Spark code runs?

Transformations are lazy and build a DAG. An action such as `count()` or `write()` makes the driver split the work into stages and tasks that run on the workers in parallel across partitions.

Why an explicit schema?

It avoids an extra data scan from `inferSchema` and prevents wrong types, for example the date as a string.

Why did you cache the DataFrame?

Seven analyses and the model reuse it, so caching avoids recomputing it each time.

Why did you exclude `units_sold` from the model?

It is recorded on the same day as demand, so it would not be known at forecast time. Using it would leak the target.

Why a time-based split?

Demand is a time series. A random split would let the model train on future days, which gives unrealistically good results.

What are your key findings?

Promotions are associated with about 30% higher demand, epidemic periods with about 38% lower demand, and discount gains level off beyond 15%.

How would this scale to 100 million rows?

The same code runs with more workers and partitions, Parquet storage and output partitioned by date.

Why did you use Google Colab and not HDFS?

The Hadoop Docker stack was not available on my machine, so I ran Spark in local mode in Colab. The Spark code is the same; only the three path variables change to run it on the project HDFS, which Student 1 and Student 2 demonstrated.

19. Conclusion

Student 3 added the distributed analytics layer to the Retail Demand Forecasting pipeline. Building on Student 1's HDFS storage and Student 2's Hive layer, Spark processed the full dataset, produced seven analyses and a forecasting model (Random Forest, RMSE 31.93, R^2 0.495), and saved the outputs. Student 4 and Student 5 can use these results for the advanced-technology, integration and presentation stages.